# CO2-AT2 SCENARIO BASED ASSESSMENT

**COURSE CODE: DSA0108** **COURSE NAME: OOPS WITH C++**

**NAME: B. DHARSHINI (192572156)**

**SIGNATURE:**

**Question 1: Hospital Consultation Charge System**

**Develop a C++ program to manage a Hospital Consultation Charge System.**

Create overloaded functions to calculate consultation charges for **General** and **Specialist** consultations.

- Use **function overloading** to calculate different consultation charges.
- Use **call by reference** to update the consultation bill.
- Use a **default argument** to include an optional service charge.
- Use **function prototyping**.
- Display the **patient type** and **total consultation charge**.

**C++ program:**

```
#include <iostream>

#include <string>

using namespace std;
```

```
// Function Prototypes
double calculateCharge(double generalCharge);
double calculateCharge(double specialistCharge, int specialist);
void updateBill(double &bill);
double addServiceCharge(double bill, double serviceCharge = 100);

int main()
{
    int choice;
    string patientName;
    double bill, finalBill;
  cout << "========== Hospital Consultation Charge System ==========\n";

    cout << "Enter Patient Name: ";
    getline(cin, patientName);

    cout << "\nPatient Type\n";
    cout << "1. General Consultation\n";
    cout << "2. Specialist Consultation\n";
    cout << "Enter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
```

```
    double charge;
    cout << "Enter General Consultation Charge: ";
    cin >> charge;

    bill = calculateCharge(charge);

    cout << "Patient Type : General Consultation\n";
}
else if (choice == 2)
{
    double charge;
    cout << "Enter Specialist Consultation Charge: ";
    cin >> charge;

    bill = calculateCharge(charge, 1);

    cout << "Patient Type : Specialist Consultation\n";
}
else
{
    cout << "Invalid Choice!";
    return 0;
}
```

```
    // Call by Reference
    updateBill(bill);

    // Default Argument
    finalBill = addServiceCharge(bill);

    cout << "\n========== Bill Details ==========\n";
    cout << "Patient Name            : " << patientName << endl;
    cout << "Consultation Charge     : " << bill << endl;
    cout << "Service Charge          : 100" << endl;
    cout << "Total Consultation Bill : " << finalBill << endl;

    return 0;
}

// Function Overloading
double calculateCharge(double generalCharge)
{
    return generalCharge;
}

double calculateCharge(double specialistCharge, int specialist)
{
    return specialistCharge + 300;
```

```
}

// Call by Reference
void updateBill(double &bill)
{
    if (bill > 1000)
    {
        bill = bill - 100;
    }
}

// Default Argument
double addServiceCharge(double bill, double serviceCharge)
{
    return bill + serviceCharge;
}
```

**Input:**

Enter Patient Name: ravi

Enter your choice: 1

Enter General Consultation Charge: 500

**Output:**

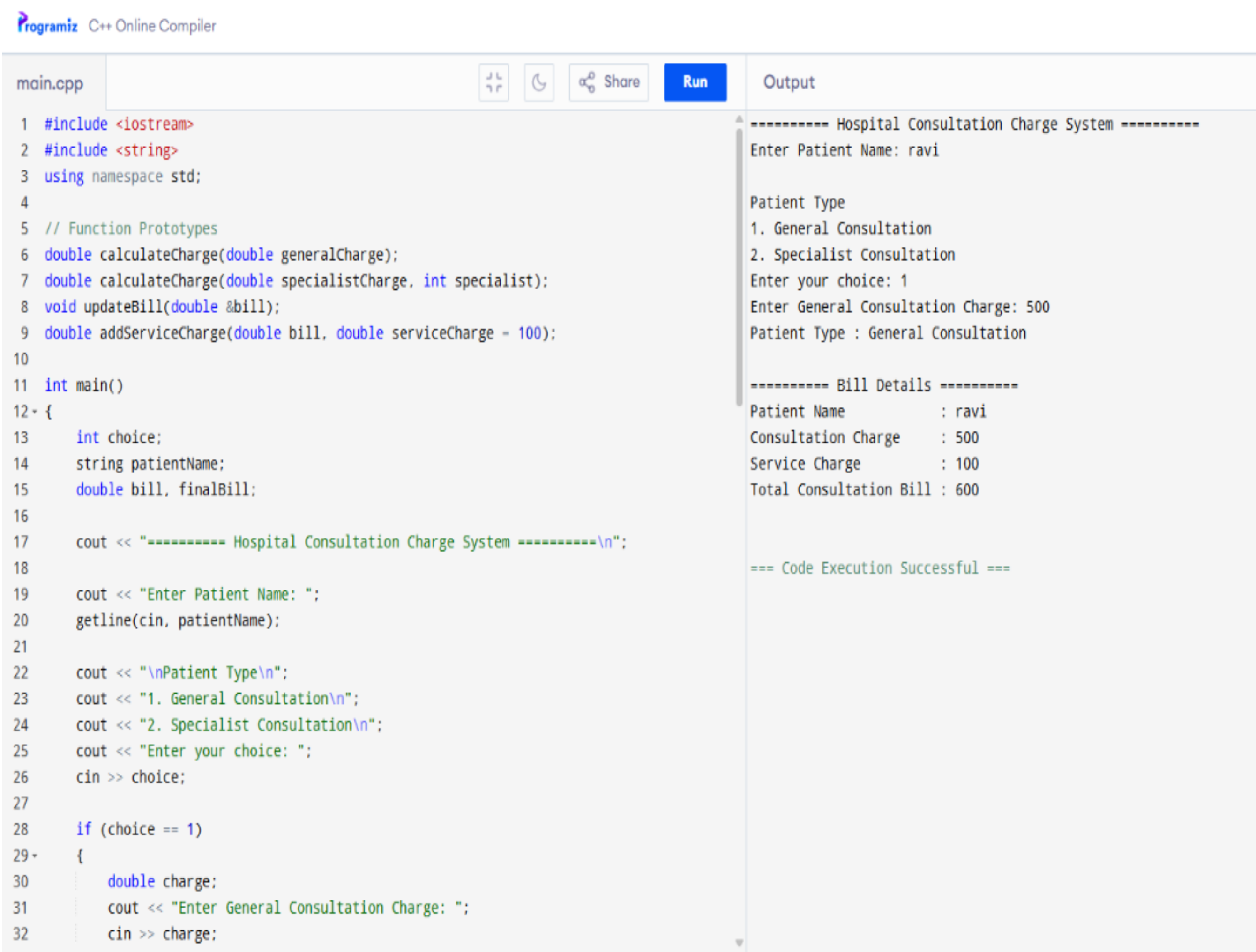
```
========== Bill Details ==========
Patient Name        : ravi
```

Consultation Charge : 500

Service Charge : 100

Total Consultation Bill: 600

Output screenshot:



# Question 2: Fitness Centre Membership System

**Develop a C++ program to manage a Fitness Centre Membership System.**

Create a class **Member** with appropriate data members such as **Member ID, Member Name, Membership Plan, and Membership Fee**.

Requirements:

- Assign membership plans and calculate the fee.
- Store member details using an **array of objects**.
- Validate the membership plan entered by the user.
- Use a **static data member** to count the total active members.
- Use a **static member function** to display the total number of active members.

**C++ program:**

```
#include <iostream>

#include <string>

using namespace std;


class Member

{

private:

    int memberID;

    string name;

    string planType;

    double fee;
```

```
    static int totalMembers;

public:
    void assignPlan();
    void calculateFee();
    void display();

    static void showTotalMembers()
    {
        cout << "\nTotal Active Members : " << totalMembers << endl;
    }
};

int Member::totalMembers = 0;

void Member::assignPlan()
{
    cout << "\nEnter Member ID: ";
    cin >> memberID;

    cin.ignore();

    cout << "Enter Member Name: ";
```

```
    getline(cin, name);

    while (true)
    {
        cout << "Enter Plan Type (Basic/Premium/VIP): ";
        getline(cin, planType);

        if (planType == "Basic" || planType == "Premium" || planType == "VIP")
            break;
        else
            cout << "Invalid Plan! Enter again.\n";
    }

    totalMembers++;
}

void Member::calculateFee()
{
    if (planType == "Basic")
        fee = 1000;
    else if (planType == "Premium")
        fee = 2000;
    else if (planType == "VIP")
```

```
        fee = 3000;
    }

void Member::display()
{
    cout << "\n----------- Membership Details -----------" << endl;
    cout << "Member ID   : " << memberID << endl;
    cout << "Member Name : " << name << endl;
    cout << "Plan Type   : " << planType << endl;
    cout << "Membership Fee : Rs. " << fee << endl;
}

int main()
{
    int n;

    cout << "========== Fitness Centre Membership System ==========\n";

    cout << "Enter Number of Members: ";
    cin >> n;

    Member m[20];
```

```
    for (int i = 0; i < n; i++)
    {
        cout << "\nEntering Details of Member " << i + 1 << endl;

        m[i].assignPlan();
        m[i].calculateFee();
    }

    cout << "\n=========== Membership Details ===========";

    for (int i = 0; i < n; i++)
    {
        m[i].display();
    }

    Member::showTotalMembers();

    return 0;
}
```

**Input:**

Enter Number of Members: 2

Enter Member ID: 101

Enter Member Name: Dharshini

Enter Plan Type (Basic/Premium/VIP): Premium

Enter Member ID: 102

Enter Member Name: Rahul

Enter Plan Type (Basic/Premium/VIP): VIP

**Output Screenshot:**

Programiz C++ Online Compiler

main.cpp Share Run

```cpp
#include <iostream>
#include <string>
using namespace std;

class Member
{
private:
    int memberID;
    string name;
    string planType;
    double fee;

    static int totalMembers;

public:
    void assignPlan();
    void calculateFee();
    void display();

    static void showTotalMembers()
    {
        cout << "\nTotal Active Members : " << totalMembers << endl;
    }
};

int Member::totalMembers = 0;

void Member::assignPlan()
{
    cout << "\nEnter Member ID: ";
    cin >> memberID;

```

Output

```
========== Fitness Centre Membership System ==========
Enter Number of Members: 1

Entering Details of Member 1

Enter Member ID: 102
Enter Member Name: ravi
Enter Plan Type (Basic/Premium/VIP): VIP

=========== Membership Details ===========
----------- Membership Details -----------
Member ID   : 102
Member Name : ravi
Plan Type   : VIP
Membership Fee : Rs. 3000

Total Active Members : 1


=== Code Execution Successful ===
```

# Question 3: Loan Repayment System

**Develop a C++ program to manage a Loan Repayment System.**

Requirements:

- Use **function prototyping**.
- Use **function overloading** to calculate repayment for different loan types.
- Use **call by reference** to update the repayment amount.
- Use **default arguments** for optional processing charges.
- Use an **inline function** to calculate interest.
- Display the loan amount, interest, and total repayment amount.

**C++ program:**

```
#include <iostream>
using namespace std;

// Function Prototypes
inline double calculateInterest(double loan, double rate);
double calculateTotal(double loan, double rate = 8.5);
void applySubsidy(double &amount);

int main()
{
    double loanAmount, rate, totalAmount;

    cout << "========== Loan Repayment System ==========\n";

    cout << "Enter Loan Amount: ";
    cin >> loanAmount;
```

```
    cout << "Enter Interest Rate (%): ";
    cin >> rate;

    double interest = calculateInterest(loanAmount, rate);

    totalAmount = calculateTotal(loanAmount, rate);

    // Call by Reference
    applySubsidy(totalAmount);

    cout << "\n========== Loan Details ==========\n";
    cout << "Loan Amount     : " << loanAmount << endl;
    cout << "Interest        : " << interest << endl;
    cout << "Payable Amount  : " << totalAmount << endl;

    return 0;
}

// Inline Function
inline double calculateInterest(double loan, double rate)
{
    return (loan * rate) / 100;
}

// Default Argument
double calculateTotal(double loan, double rate)
{
    return loan + calculateInterest(loan, rate);
```

```
}

// Call by Reference
void applySubsidy(double &amount)
{
  if (amount > 100000)
  {
    amount = amount - 5000;
  }
}
```

**Input:**

Enter Loan Amount: 150000

Enter Interest Rate (%): 10

**Sample output:**

```
========== Loan Repayment System ==========

Enter Loan Amount: 150000
Enter Interest Rate (%): 10

========== Loan Details ==========
Loan Amount     : 150000
Interest        : 15000
Payable Amount  : 160000
```

Output screenshot:

Programiz C++ Online Compiler

main.cpp

```cpp
#include <iostream>
using namespace std;

// Function Prototypes
inline double calculateInterest(double loan, double rate);
double calculateTotal(double loan, double rate = 8.5);
void applySubsidy(double &amount);

int main()
{
    double loanAmount, rate, totalAmount;

    cout << "========== Loan Repayment System ==========\n";

    cout << "Enter Loan Amount: ";
    cin >> loanAmount;

    cout << "Enter Interest Rate (%): ";
    cin >> rate;

    double interest = calculateInterest(loanAmount, rate);

    totalAmount = calculateTotal(loanAmount, rate);

    // Call by Reference
    applySubsidy(totalAmount);

    cout << "\n========== Loan Details ==========\n";
    cout << "Loan Amount      : " << loanAmount << endl;
    cout << "Interest         : " << interest << endl;
    cout << "Payable Amount   : " << totalAmount << endl;
```

Output

```
========== Loan Repayment System ==========
Enter Loan Amount: 150000
Enter Interest Rate (%): 10

========== Loan Details ==========
Loan Amount      : 150000
Interest         : 15000
Payable Amount   : 160000


=== Code Execution Successful ===
```

## Question 4: Parcel Charge Management System

**Develop a C++ program to manage a Parcel Charge Management System.**

Create a class **Parcel** with appropriate data members such as **Parcel Type, Weight, Delivery Distance, and Packaging Charge**.

Requirements:

- Use **function overloading** to calculate charges for **Normal** and **Express** delivery.
- Use a **default argument** for packaging charges.
- Use an **inline function** to calculate the base delivery charge.
- Use a **static data member** to count the total number of parcels processed.
- Use a **static member function** to display the total number of parcels processed.

C++ program:

```
#include<iostream>

#include<string>

 using namespace std;

class Parcel

 {

 private:

string parcelType;

float weight;

float distance;

float packagingCharge;

static int totalParcels;
```

```
public:

 void readDetails();

 inline float baseCharge();

float calculateCharge(float packageCharge = 50); // Normal Delivery

 float calculateCharge(int express, float packageCharge = 50); // Express Delivery

void display(float total);

static void showTotalParcels();


};

int Parcel::totalParcels = 0;

// Read Parcel Details

 void Parcel::readDetails()

 {

 cout << "Enter Parcel Type: "; cin >> parcelType;

cout << "Enter Parcel Weight (kg): ";
cin >> weight;

cout << "Enter Delivery Distance (km): ";
cin >> distance;

cout << "Enter Packaging Charge (Enter 0 for default): ";
cin >> packagingCharge;

if (packagingCharge == 0)
    packagingCharge = 50;

totalParcels++;


}
```

```
// Inline Function

 inline float Parcel::baseCharge()

 {

 return (weight * 20) + (distance * 5);

}

// Function Overloading - Normal Delivery

 float Parcel::calculateCharge(float packageCharge)

 {

return baseCharge() + packageCharge;

 }

// Function Overloading - Express Delivery

float Parcel::calculateCharge(int express, float packageCharge)

 {

return baseCharge() + packageCharge + 100;

 }

// Display Details void Parcel::display(float total)

{

 cout << "\n--------- Parcel Details ---------\n"; cout << "Parcel Type : " << parcelType << endl;
cout << "Weight : " << weight << " kg" << endl; cout << "Distance : " << distance << " km" <<
endl;

 cout << "Packaging Charge : Rs. " << packagingCharge << endl; cout << "Total Charge : Rs. "
<< total << endl; }

// Static Member Function

void Parcel::showTotalParcels()
```

```
 {
 cout << "\nTotal Parcels Processed: " << totalParcels << endl;
}
int main()
{
 Parcel p;
int choice;
 float totalCharge;
cout << "========== Parcel Charge Management System ==========\n";

p.readDetails();

cout << "\nDelivery Type\n";
cout << "1. Normal Delivery\n";
cout << "2. Express Delivery\n";
cout << "Enter Choice: ";
cin >> choice;

if (choice == 1)
    totalCharge = p.calculateCharge();
else if (choice == 2)
    totalCharge = p.calculateCharge(1);
else
{
    cout << "Invalid Choice!";
    return 0;
}

p.display(totalCharge);

Parcel::showTotalParcels();

return 0;
}
```

Input:

Enter Parcel Type: Box

Enter Parcel Weight (kg): 5

Enter Delivery Distance (km): 20

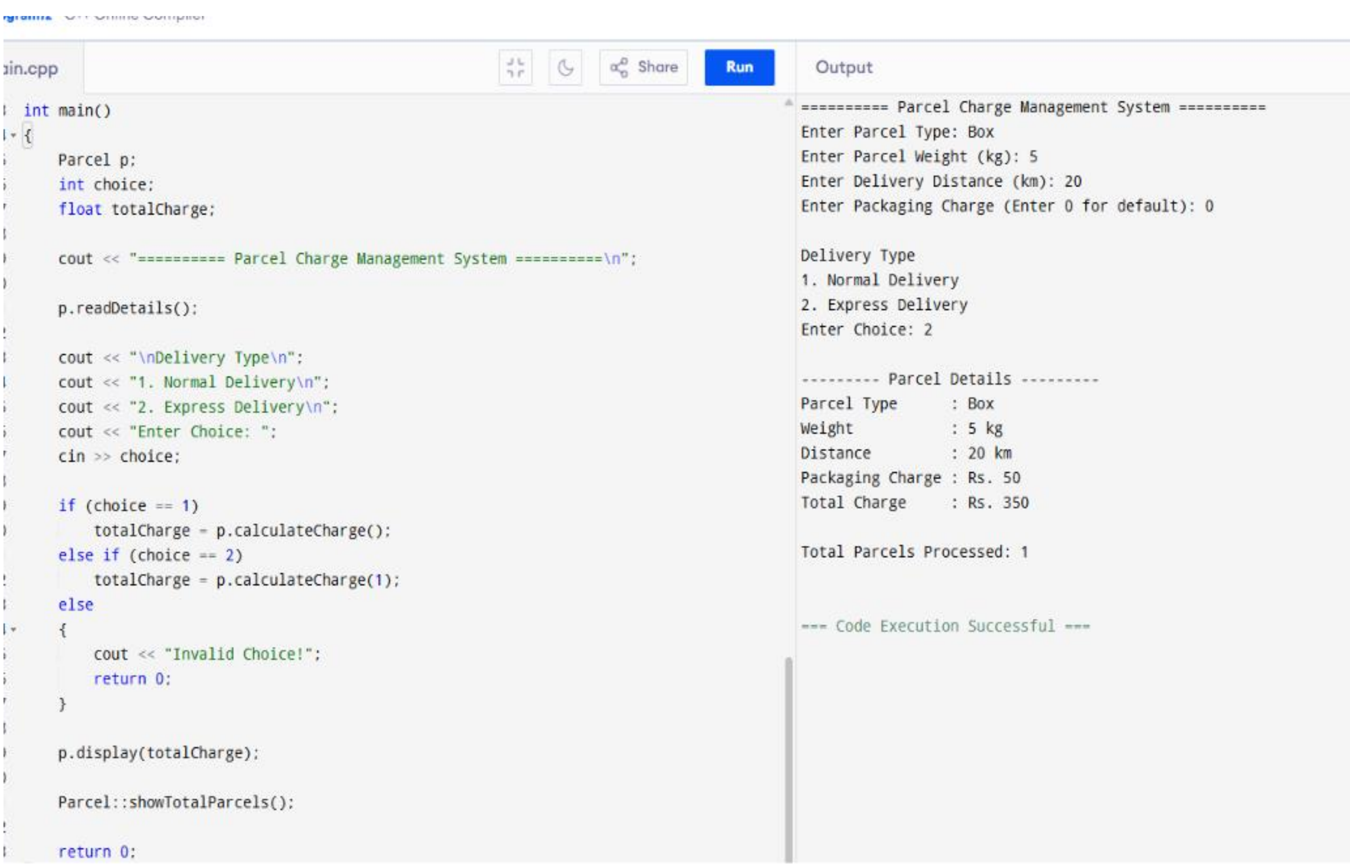
Enter Packaging Charge (Enter 0 for default): 0

Delivery Type

1. Normal Delivery

2. Express Delivery

Enter Choice: 2



```cpp
int main()
{
    Parcel p;
    int choice;
    float totalCharge;

    cout << "========== Parcel Charge Management System ==========\n";

    p.readDetails();

    cout << "\nDelivery Type\n";
    cout << "1. Normal Delivery\n";
    cout << "2. Express Delivery\n";
    cout << "Enter Choice: ";
    cin >> choice;

    if (choice == 1)
        totalCharge = p.calculateCharge();
    else if (choice == 2)
        totalCharge = p.calculateCharge(1);
    else
    {
        cout << "Invalid Choice!";
        return 0;
    }

    p.display(totalCharge);

    Parcel::showTotalParcels();

    return 0;
```

Output

```
========== Parcel Charge Management System ==========
Enter Parcel Type: Box
Enter Parcel Weight (kg): 5
Enter Delivery Distance (km): 20
Enter Packaging Charge (Enter 0 for default): 0

Delivery Type
1. Normal Delivery
2. Express Delivery
Enter Choice: 2

--------- Parcel Details ---------
Parcel Type      : Box
Weight           : 5 kg
Distance         : 20 km
Packaging Charge : Rs. 50
Total Charge     : Rs. 350

Total Parcels Processed: 1


=== Code Execution Successful ===
```

# Question 5: Water Billing System

**Develop a C++ program to manage a Water Billing System.**

Create a class **WaterBill** with appropriate data members such as **Consumer Number, Consumer Name, Water Units Consumed, and Bill Amount**.

Requirements:

- Use a **constructor** to initialize the data members.
- Use **function overloading** to calculate the water bill.
- Use a **static data member** to count the total number of consumers.
- Use a **static member function** to display the total number of consumers.
- Display the **consumer details** and **total water bill**.

C++ program:

```
#include<iostream>

#include<string>

 using namespace std;

class WaterBill

{

 private:

 int consumerNo;

 string consumerName;

 float units;

float billAmount;

static int totalConsumers;
```

```
public:

 // Constructor

 WaterBill()

 {

 consumerNo = 0;

 consumerName = "";

units = 0;

 billAmount = 0;

 }

void getDetails();

// Function Overloading
void calculateBill();
void calculateBill(float extraCharge);

void display();

static void showTotalConsumers()
{
    cout << "\nTotal Consumers Processed: " << totalConsumers << endl;
}


};

// Static Member Initialization int WaterBill::totalConsumers = 0;

// Get Consumer Details void WaterBill::getDetails() { cout << "Enter Consumer Number: "; cin
>> consumerNo;

cin.ignore();

cout << "Enter Consumer Name: ";
getline(cin, consumerName);
```

```
cout << "Enter Water Units Consumed: ";
cin >> units;

totalConsumers++;


}

// Calculate Bill (Normal)

void WaterBill::calculateBill()

 {

if (units <= 100)

          billAmount = units * 2;

 else if (units <= 200)

            billAmount = (100 * 2) + ((units - 100) * 3);

 Else

           billAmount = (100 * 2) + (100 * 3) + ((units - 200) * 5);

}

// Calculate Bill (With Extra Charge)

void WaterBill::calculateBill(float extraCharge)

 {

 calculateBill(); billAmount += extraCharge;

 }

// Display Details

void WaterBill::display()

 {
```

```
cout << "\n========== Water Bill ==========\n"; cout << "Consumer Number : " <<
consumerNo << endl;

cout << "Consumer Name : " << consumerName << endl;

 cout << "Units Consumed : " << units << endl;

cout << "Total Bill : Rs. " << billAmount << endl;

 }

int main()

 {

 WaterBill w;

int choice;

cout << "========== Water Billing System ==========\n";

w.getDetails();

cout << "\n1. Normal Bill\n";
cout << "2. Bill with Service Charge\n";
cout << "Enter Choice: ";
cin >> choice;

if (choice == 1)
{
    w.calculateBill();
}
else if (choice == 2)
{
    float extra;
    cout << "Enter Service Charge: ";
    cin >> extra;
    w.calculateBill(extra);
}
else
{
    cout << "Invalid Choice!";
```

```
    return 0;
}

w.display();

WaterBill::showTotalConsumers();

return 0;


}
```

**Input:**

========== Water Billing System ==========

Enter Consumer Number: 101

Enter Consumer Name: Dharshini

Enter Water Units Consumed: 250

1. Normal Bill
2. Bill with Service Charge

Enter Choice: 2

Enter Service Charge: 50

Output screenshot:

Programiz C++ Online Compiler

main.cpp Share Run

```cpp
#include <iostream>
#include <string>
using namespace std;

class WaterBill
{
private:
    int consumerNo;
    string consumerName;
    float units;
    float billAmount;

    static int totalConsumers;

public:
    // Constructor
    WaterBill()
    {
        consumerNo = 0;
        consumerName = "";
        units = 0;
        billAmount = 0;
    }

    void getDetails();

    // Function Overloading
    void calculateBill();
    void calculateBill(float extraCharge);

    void display();
```

Output

```
---------- Water Billing System ----------
Enter Consumer Number: 102
Enter Consumer Name: kirthi
Enter Water Units Consumed: 250

1. Normal Bill
2. Bill with Service Charge
Enter Choice: 2
Enter Service Charge: 50

---------- Water Bill ----------
Consumer Number  : 102
Consumer Name    : kirthi
Units Consumed   : 250
Total Bill       : Rs. 800

Total Consumers Processed: 1


=== Code Execution Successful ===
```